intensity of 357 gCO_2 per kWh[101].

$$\begin{aligned} &65 \text{ days} \times 24 \text{ hours} \times 800 \text{ A100s (80GB)} \\ &\times .4 \text{ kW TDP} \times 1.1 \text{ PUE} \approx 549.1 \text{ MWh} \\ &\times 357 \text{ gCO}_2 \text{ per kWh} \approx 196.0 \text{ mtCO}_2 \end{aligned}$$

BLOOM. We note that the authors of Bloom are still calculating carbon emissions, in the interim we use an estimation method aligned with the above approaches. Bloom took roughly 117 days to train the model on 384 A100s.[102] The model was trained in France with an average carbon intensity of 64 gCO_2 per kWh in 2020.[103]

$$\begin{aligned} &117 \text{ days} \times 24 \text{ hours} \times 384 \text{ A100s (80GB)} \\ &\times .4 \text{ kW TDP} \times 1.1 \text{ PUE} \approx 474.4 \text{ MWh} \\ &\times 64 \text{ gCO}_2 \text{ per kWh} \approx 30.4 \text{ mtCO}_2 \end{aligned}$$

Caveats. Since many details are missing to estimate the true power draw and there are significant difficulties in achieving accurate estimates, the figures here should only be thought of as a rough estimate. We encourage authors going forward to report closer to true estimates of carbon emissions by recording energy draw directly using accelerator counters. We also note that ours may be an underestimate since we have no information on CPUs used, which also contribute to energy consumption (even though the PUE factor is meant to capture this). Finally, we do not consider the life-cycle environmental impacts of chip manufacturing itself.

D Perturbations

D.1 Robustness

Lowercase. We transform all the text of an instance into lowercase.

Contractions/Expansions. These two perturbations are based on the `Contractions and Expansions` perturbation from NL-Augmenter (Dhole et al., 2021). These are commonly used forms such as "I'm / I am" or "She is / She's". In contraction, we replace all occurrences of the pre-defined list of each expansion with their contracted versions. Similarly, Expansion perturbation replaces all occurrences of all contractions with their expanded versions.

D.1.1 Individual perturbations

Our focus is on perturbations that are relatively natural in the context of users interacting with text interfaces. First, we describe the individual perturbations that we consider, and in the next section we describe how we use them as building blocks to compose other perturbations.

Misspellings. The misspelling perturbation is inspired by the `Correct Common Misspellings` perturbation from NL-Augmenter (Dhole et al., 2021). The common misspellings are based on Wikipedia's List of Common Misspellings, which maps misspellings to correct words. We invert this mapping to produce a dictionary that maps a correctly spelled word to its common misspellings. We implement this perturbation by iterating through each word in the input, searching for whether we have common misspellings for that word, then with some probability (independently for each word), replace that word with a random common misspelling. Plural forms of words are handled by augmenting the dictionary of common misspellings to include plural forms.

[101] `https://app.electricitymaps.com/zone/RU-1`.
[102] `https://huggingface.co/bigscience/bloom/blob/main/README.md`.
[103] `https://app.electricitymaps.com/zone/FR`.

Extra spaces. We replace each sequence of space (" ") characters with another sequence of space characters of random length (typically this just adds extra space between words and sentences).

D.1.2 Composing perturbations

Given the basic perturbations described above, we compose them to create a composite perturbation. This allows us to evaluate model robustness in a scalable manner, as opposed to evaluating models on each individual perturbation for each instance. Concretely, our overall perturbation recipe for the core experiments is the following:

- Transform the instance into lowercase (e.g., "Hello" → "hello")
- Apply the contraction perturbation (e.g., "it is" → "it's")
- Attempt to misspell each word with probability 0.1 (e.g., "the"→"teh") (note that we don't have misspellings for all possible words, so the actual probability will be lower in practice)
- Replace each space with 1 to 3 space characters randomly (e.g., "hi ␣ there"→"hi ␣ ␣ ␣ there")

D.2 Fairness

We perturb evaluation instances using the following perturbation methods. Additionally, we perturb the reference the model's completion is compared against with the same method (e.g. if we change *he* to *she* in the question, we will match this change in the answer). For all perturbations, we apply them deterministically whenever possible. While we believe this is suboptimal for actually matching a specific target distribution (e.g. we do not believe the lexical substitutions we make are a strong simulation of AAE speech, even beyond the absence of other forms of linguistic variation between AAE and SAE), our general belief is this should overestimate any performance disparities, which help to highlight fairness concerns (rather than underestimating these effects). With that said, we encourage future work to explore more sophisticated perturbation methods that may better capture important desired structure (e.g. properly model coreference in the swapping of gender pronouns, or reflect other linguistic properties of AAE speech (Ziems et al., 2022)).

1. Dialect Perturbation: We currently support conversions between Standard American English (SAE) and African American English (AAE) using the mapping between lexical terms provided by Ziems et al. (2022).
2. Gender Pronoun Perturbation: We support conversions between the gender neutral and gendered pronouns from Lauscher et al. (2022).
3. Gender Term Perturbation: We convert gender terms of a source gender (e.g. "Grandfather") to their counterparts in a target gender (e.g. "Grandmother"). We build our mapping by improving the union of the mappings from Garg et al. (2018) and Bolukbasi et al. (2016).
4. First Name Perturbation: We convert first names in a source race or gender to those in the target race or gender, using the names from Caliskan et al. (2017), which derives its list form Greenwald et al. (1998). The associations between demographic category and name are derived from US Census statistics; we note that these statistical relationships may also not be invariant across time (and hence may be less valid when considering text from different periods of time). That is, the statistics used reify a particular set of associations that are likely to change as a function of time due to cultural changes (especially the relationship between self-identified gender/race and names as the categorizations themselves change with time), population changes (e.g. immigration), and political changes (e.g. the Chinese Exclusion Act).
5. Last Name Perturbation: We convert last names in a source race to those in the target race, using the last names from Garg et al. (2018), which derives its list form Chalabi & Flowers (2017). See the above discussion of the relationship between names and demographic information; we also note that the frequent instance of name change through marriage is poorly addressed in the case of last names (i.e. an implicit codification of intra-racial marriage).

E Targeted evaluations

E.1 Language

E.1.1 The Pile

Scenario Description. **The Pile** (Gao et al., 2021a) is a language modeling scenario that includes large-scale English texts from a diverse list of domains. As an example, an input for the OpenSubtitle (Tiedemann, 2016) subset of the scenario looks like:

> "It came from down here." "What were you thinking bringing a stranger here?" "... look out for herself." "I wouldn't be alive if it wasn't for her." "Yeah, well, I'm protecting you now."

The textual output of a language model should be the same with the input. The main metric for the scenario is bits per byte (BPB).

Data. Gao et al. (2021a) created the dataset by incorporating or extending high-quality existing datasets and by filtering or extracting from a few raw data sources. The total size of the dataset is 825GB, and each of the development and test sets is a random sample containing 0.1% of the data. We only use the subsample of the test set (one-tenth of the whole test set) used in Gao et al. (2021a) for evaluation.

Pre-processing. There is no scenario-level pre-processing. To adapt the scenario for each model, we follow (Gao et al., 2021a) and split each instance (passage) to non-overlapping token chunks whose length equal the max input length of the model.

Data Resources. We access the dataset at `https://pile.eleuther.ai/`. The dataset is made available through our benchmark.

E.1.2 BLiMP

Scenario Description. **BLiMP** (Warstadt et al., 2020) is a linguistic minimal pair scenario that evaluates the linguistic knowledge of language models on a wide range of grammatical phenomena in English. The scenario consists of 67 subsets which cover 12 categories of syntactic, morphological, or semantic phenomena. Each subset contains 1,000 minimal pairs.

As an example, a pair of inputs for the scenarios look like:

> Good: Craig explored that grocery store. Bad: Craig explored that grocery stores.

If the model predicts a higher log-probability-per-token for the good input, it is considered correct. The accuracy for this scenario is measured using exact match.

Data. Warstadt et al. (2020) used linguist-crafted grammar templates to generate the datasets and had human annotators verify the labels. BLiMP only consists of a test split, with inputs on average being shorter than 10 tokens based on the GPT-2 tokenizer.

Pre-processing. We cast the problem as multiple choice with the good and bad sentences being the two choices.

Data Resources. We access the dataset at `https://github.com/alexwarstadt/blimp`. The dataset is made available through our benchmark. The data generation toolbox that was used to generate BLiMP can be found at `https://github.com/alexwarstadt/data_generation`.